

SQLALCHEMY

I. Fondamentaux - connexion

II. ORM - modèles

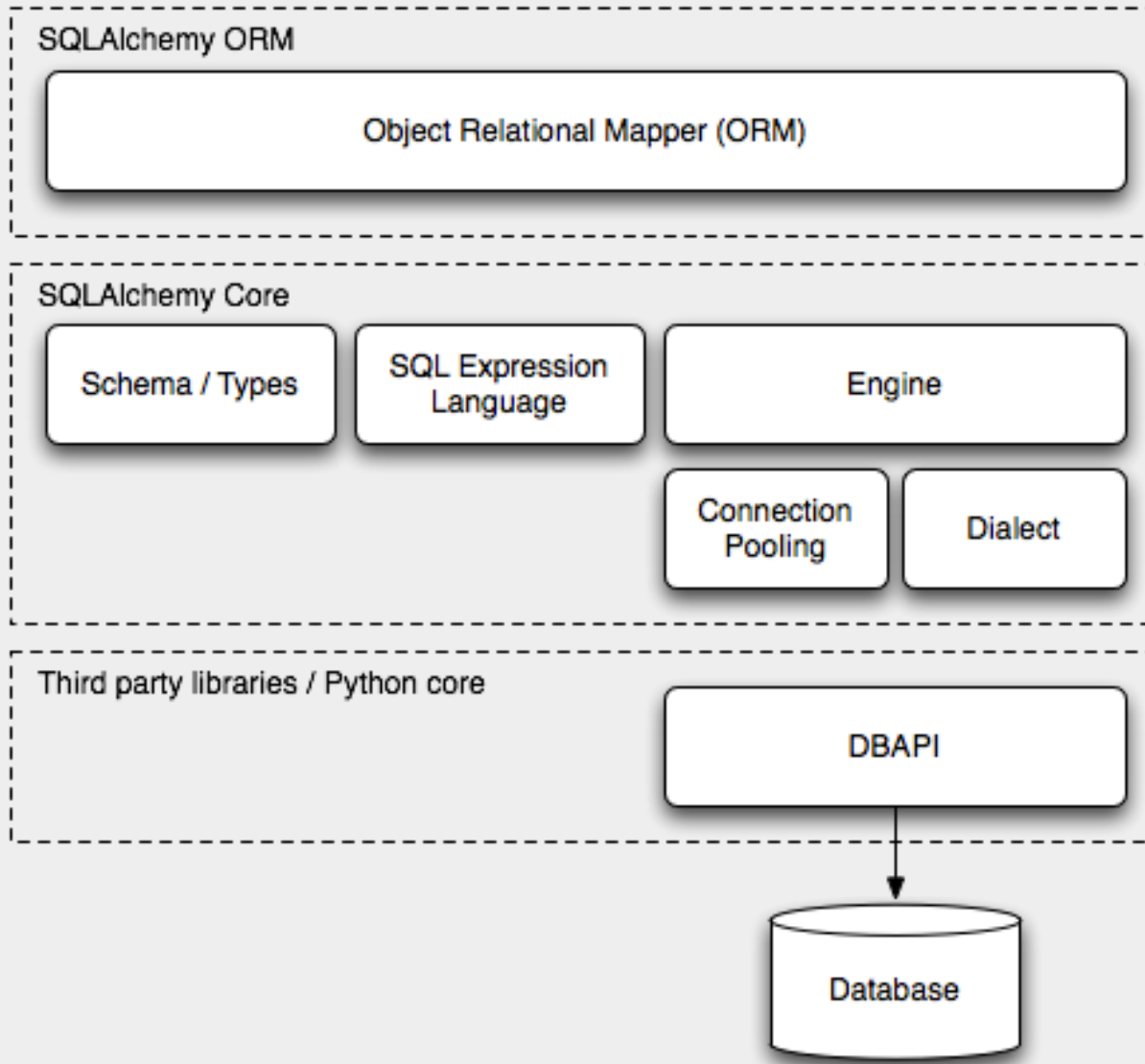
III. ORM - persistance

IV. CRUD

V. Migration avec Alembic

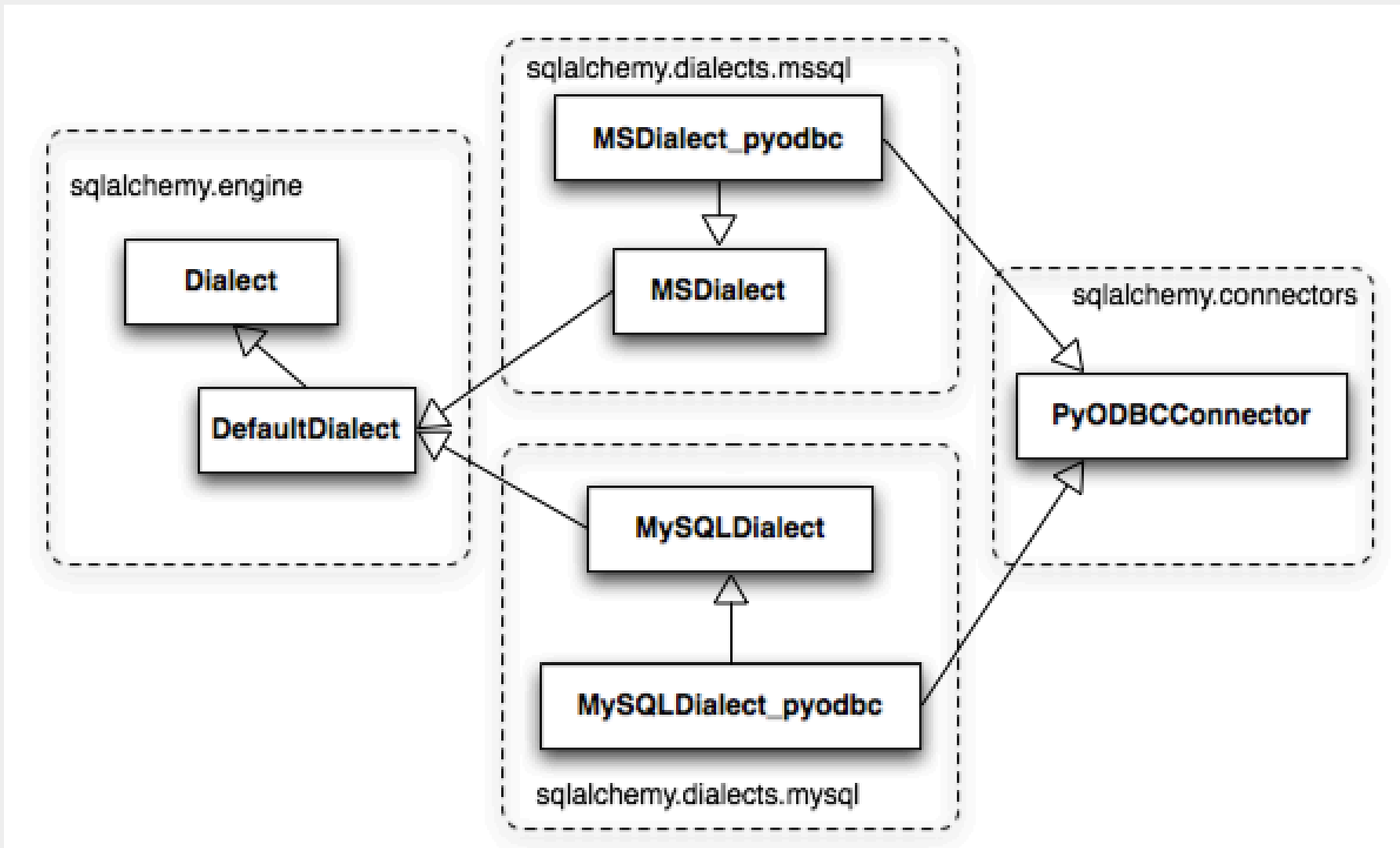
I. Fondamentaux - connexion

structure





interfaces DBAPI Vs PyODBC



pool de connexions

- l'engine gère une instance de `QueuePool` pour gérer plusieurs connexions de façon concurrente

```
from sqlalchemy import create_engine
engine = create_engine(URL,
    pool_size=10, # cnx "idle", réutilisables dans le pool
    max_overflow=20 # cnx supplémentaires mais détruites après usage
)

# pas de pool, une cnx par requete
# en cas de pool côté serveur (ex: pgBouncer)
engine = create_engine(URL, pool_class=NullPool)
```

Niveaux d'Isolation

- en cas de transactions concurrentes, pour éviter les collisions de lectures/écritures comme les
- *dirty reads* => lecture de données non validées
- *non-repeatable reads* => lecture de données modifiées par une autre transaction
- *phantom reads* => lecture de données insérées par une autre transaction

```
# MAIS COUTE EN PERFORMANCE !!!  
# ... ou ne pas créer de transactions trop longues  
engine = create_engine(URL, isolation_level="SERIALIZABLE")
```


Session: espace de travail

- pour les *objets persistants* et les *transactions*
- une session peut être générée à partir d'une factory **sessionmaker**

```
# à placer dans un module global
from sqlalchemy.orm import sessionmaker
SessionLocal = sessionmaker(
    bind=engine,
    autocommit=False, # commit explicite par transaction
    autoflush=True, # flush implicite cf intra
)
```

Session: transaction

- les transactions sont intégrés à la session via les gestionnaires de contexte

```
with SessionLocal(engine) as session:
    with session.begin():
        # try:
        session.add(some_object)
        # except:
        #     session.rollback()
        # else:
        #     session.commit()
```

session: en dehors d'un gestionnaire de contexte

```
from typing import Generator
from sqlalchemy.orm import Session

# pour réaliser une injection de dépendance
#                               yielded sent returned
def get_db() -> Generator[Session, None, None]:
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

II. ORM - modèles

Description déclarative

- permet de piloter la création de tables à partir de classes Python utilisant des **annotations de types**

```
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column
from sqlalchemy import String
from typing import Optional
class Base(DeclarativeBase):
    pass

class User(Base):
    __tablename__ = "users"
    id: Mapped[int] = mapped_column(primary_key=True)
    # NULLABLE << NOT NULL !!
    name: Mapped[Optional[str]] = mapped_column(String(50), nullable=False)
```

Relations

```
from sqlalchemy import ForeignKey
from sqlalchemy.orm import relationship

class Person(Base):

    # champs SQL: unique=True => OneToOne, sinon OneToMany
    user_id: Mapped[int] = mapped_column(
        ForeignKey("users.id", ondelete="SET NULL"),
        nullable=True, unique=True)

    # attribut Python
    user: Mapped["User"] = relationship(
        back_populates="person")
```

relation ManyToMany

```
# style impératif
role_permission = Table(
    'role_permissions', Base.metadata,
    Column('role_id', ForeignKey('roles.id'), primary_key=True),
    Column('permission_id', ForeignKey('permissions.id'), primary_key=True)
)
# ... dans Role
permissions: Mapped[List["Permission"]] = relationship(
    secondary=role_permission,
    back_populates="roles"
)
# ... et dans Permission
roles: Mapped[List["Role"]] = relationship(
    secondary=role_permission,
    back_populates="permissions"
)
```

cascade avec python

- le paramètre `cascade` de `relationship` permet de propager les opérations de persistance **dans le sens d'une clé étrangère** comme:
 - *save-update* (par défaut) : persiste les objets enfants
 - *delete* : supprime les objets enfants
 - *delete-orphan* : supprime les objets enfants même supprimés du parent

cascade événementielle

- en cas de cascade *en sens inverse* (enfant -> parent)
- on peut utiliser le décorateur `@event.listens_for`
 - pour associer une fonction à un événement de cycle de vie d'un objet (ex. `before_delete`)

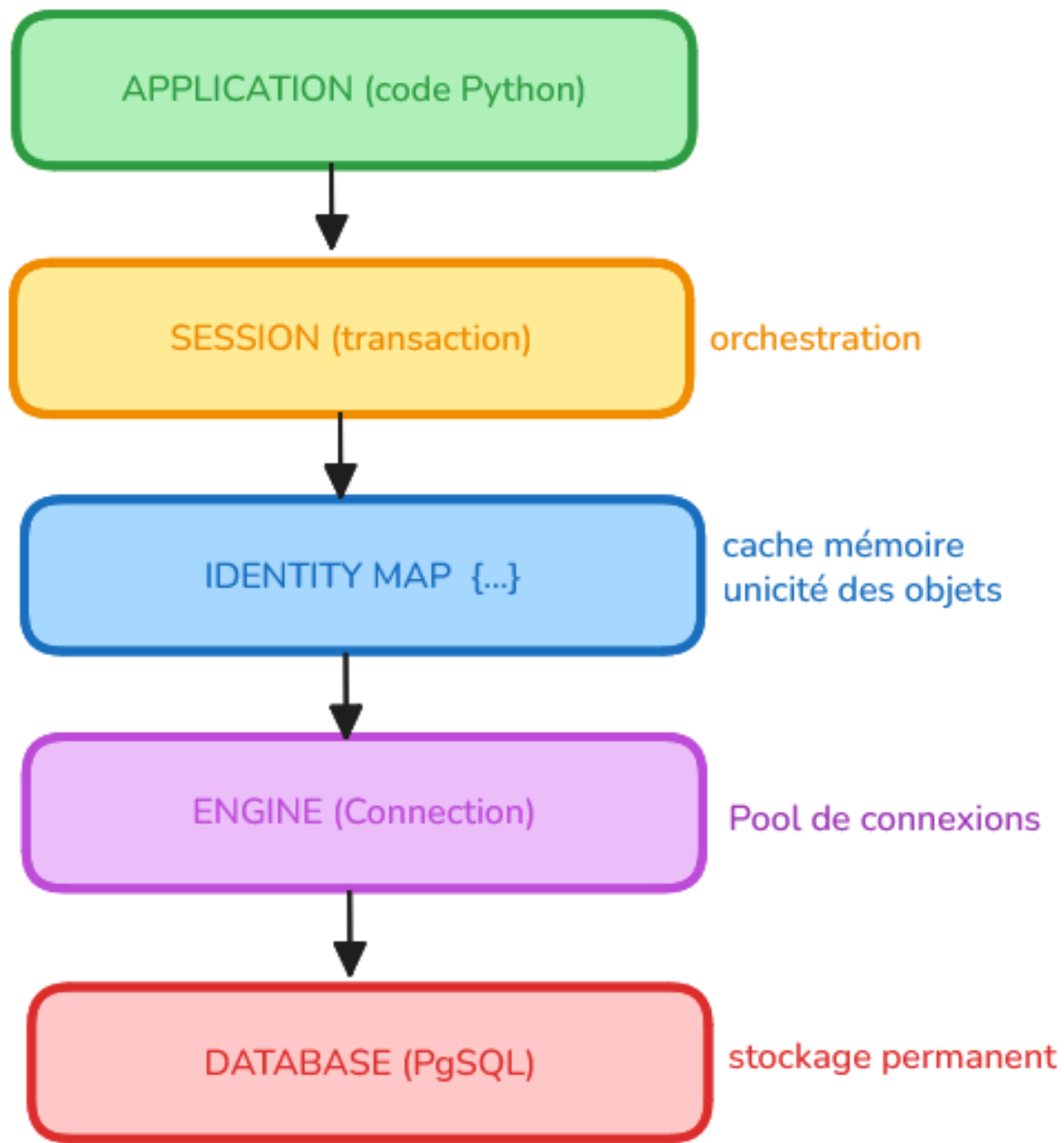
```
@event.listens_for(Model, "before_delete")
def hook(mapper, connection, target):
    # avant de supprimer l'objet target
    # faire quelque chose avec la connection => session
```

creation du schéma à partir des modèles

```
Base.metadata.drop_all(bind=engine)
# CREATE TABLE IF NOT EXISTS
Base.metadata.create_all(bind=engine)

# ou table par table
User.__table__.drop(engine)
User.__table__.create(engine)
```

III. ORM - persistance



Identity Map - Le cache de session

- cache mémoire qui garantit l'unicité d'un enregistrement de la base dans une session.

```
# map vide  
stmt = select(User).where(User.id == 1)  
# SELECT * FROM users WHERE id=1  
user1 = db.execute(stmt).scalar()  
# user1 est dans le map => {(User, 1): obj}  
# donc pas de SQL et user2 is user1  
user2 = db.execute(stmt).scalar()
```

flush()

- synchroniser les changements de la session avec la base **avant commit**
- **après le flush** les requêtes suivantes voient ces changements *nouveaux objets, attributs modifiés, objets supprimés*

```
user1.name = "new name"
db.execute(select(User).filter_by(id=1)).scalar().name # 'old name' !

db.flush() # génère: UPDATE users SET name='new name' WHERE id=1

db.execute(select(User).filter_by(id=1)).scalar().name # 'new name' !
db.commit() # COMMIT transaction: écrit en base, vu des autres sessions
db.close() # vide le map !
```

refresh() et expire()

- pour rafraîchir un objet avec les données de la base

```
# user1 est dans le map, mais les données sont périmées
```

```
# génère un SELECT pour récupérer les données à jour  
db.refresh(user1)
```

```
# marque les données comme périmées, le prochain accès génère un SELECT  
db.expire(user1)
```

Session - Orchestration des opérations

- cycle de vie des objets *transient, pending, persistent, detached*
- maintient une transaction active
- coordonne les opérations avec l'Identity Map et la base de données

```
from sqlalchemy import inspect

new_user = User(name="Alice") # pas dans la session
inspect(new_user).transient # True
db.add(new_user)              # Ajoute à la session
inspect(new_user).pending # True
db.flush()                    # Flush (pending -> persistent)
db.commit()                   # COMMIT transaction (pending -> persistent)
db.close()                    # Ferme la session (persistent -> detached)
```


autoflush et autocommit

- flush implicite après certaines requêtes / changements orm pour garantir la cohérence des données
 - pourquoi *désactiver l'autoflush* ?:
 - **import massifs** de données
 - **Contrôle fin** sur le timing des requêtes SQL (ex. tests unitaires)
- “ **⚠ une session en autocommit ne gère pas de transaction, pas de rollback !** ”

IV. CRUD

insert

```
from sqlalchemy import insert

users = [User(name="Alice"), User(name="Bob")]
# génère une requête INSERT ... SELECT ... RETURNING
db.add_all(users)

# plus rapide: transformer les objets en dict et retourner les ids générés
# INSERT ... RETURNING
insert_stmt = insert(User).returning(User.id)
rets = db.execute(insert_stmt, [{"name": "Alice"}, {"name": "Bob"}]).all()
```

Select: champs

```
from sqlalchemy import select, func, text

stmt = select(User)
stmt = select(User.name, User.id)

# fonction
stmt = select(func.count(User.id))
# raw SQL
stmt = select(text("MAX(id)").select_from(User)
```

Select: filtres

```
from sqlalchemy import select
# filtre par attribut = valeur
stmt = select(User).filter_by(name="Alice")

# filtre par attribut opérateur valeur
stmt = select(User).where(User.name == "Alice")
stmt = select(User).where(User.name.in_(("Alice", "Bob")))
```

select: group by

```
from sqlalchemy import select, func
stmt = select(
    User.name,
    func.count(User.id).label("count")
).group_by(User.name)
# having
stmt = stmt.having(func.count(User.id) > 1)
```

select: order_by, limite et offset

```
from sqlalchemy import select  
stmt = select(User).limit(10).offset(20).order_by(User.name.desc())
```

select: jointures

```
# EAGER LOADING
from sqlalchemy.orm import joinedload, selectinload
many2one = select(Person).options(joinedload(Person.user))
one2many = select(User).options(selectinload(User.person))

imbricated = select(User).options(
    selectinload(User.person),
    joinedload(Person.user))

explicit = select(
    Address.city, func.count(Person.id).label('count')
).join(Address, Address.person_id == Person.id
).group_by(Address.city)

none_results = (
    select(User)
    .outerjoin(Person.user)
    .where(Person.id.is_(None)))
```


jointures de relations

```
user: Mapped["User"] = relationship(lazy="<option>")
```

- **select** => jointure différée au premier accès (*LAZY par défaut*)
 - **joined** => jointure **LEFT OUTER | INNER** (*EAGER*)
 - **selectin** => jointure **WHERE xxx IN (...)** (*EAGER*)
 - **raise** => jamais de jointure sinon exception
- “ *comportement par défaut de la relation.*
à changer le cas échéant avec une **jointure avec select** ”

Recommandations

Situation	type	Raison
Relations rarement utilisées	select	Économise ressources
Toujours besoin des relations	joined	Une seule requête
Charger liste d'objets	selectin	Évite N+1, one2many
Petites collections simples	joined	Simple et efficace
Relations complexes/volumineuses	select + explicite	Contrôle manuel

le pb des N+1 requêtes

- ⚠ en cas de chargement différé (LAZY), une requête est générée à chaque accès à la relation

```
users = db.execute(select(User)).scalars().all()
for user in users:
    print(user.person.name) # génère une requête par user
```

hydratation des résultats

- **"fetchone"** => un seul résultat ou exception ou None
 - `obj_any = db.execute(stmt).scalar_one_or_none()`
 - `obj_one = db.execute(stmt).scalar_one()` *(exception)*
 - `value = db.execute(stmt).scalar()` *(une seule colonne)*
 - `row = db.execute(stmt).first()` *(+ léger)*
- **"fetchall"** => liste de résultats
 - `objs = db.execute(stmt).scalars().all()`
 - `rows = db.execute(stmt).all()`

Update

```
from sqlalchemy import update
stmt = update(User).where(User.name == "Alice").values(name="Alicia")

# par défaut, update et delete rafraichissent les objets affectés
# inutile à la fin de la session/transaction donc
stmt = stmt.execution_options(synchronize_session=False)

# champs à retourner
db.execute(stmt.returning(User.name)).all()

# ou nb de lignes affectées
db.execute(stmt).rowcount
```

V. Migration avec Alembic

procédure Alembic: install & config

1. `pip install alembic`
2. `alembic init alembic` => dossier *alembic/* + fichier **alembic.ini**
3. config bdd dans `alembic.ini`
4. configurer `alembic/env.py` pour **importer les models**

procédure Alembic: migration

1. faire une modification du modèle

2. fait le **delta entre l'état du python et la bdd**

```
alembic revision --autogenerate -m "<description>"
```

3. `alembic upgrade head` : *applique la migration*

- et génère une table de versionnement `alembic_version` pour suivre les migrations appliquées

autres commandes utiles

```
alembic history [--verbose] # liste les migrations  
alembic downgrade <rev> | -n | <rev>-n # annule une ou plusieurs migrations  
alembic upgrade <rev> | +n | <rev>+n # applique une ou plusieurs migrations  
  
alembic current # affiche la migration courante  
alembic show <rev> # affiche le contenu d'une migration
```

VI. sqlalchemy en asynchrone

pourquoi le faire ?

- sqlalchemy gère des requêtes exécutées dans des threads.
si une requête effectue des opérations d'entrée/sortie (i/o) bloquantes, ex: appels à une base de données
- en utilisant les outils asynchrones et leurs coroutines, les opérations i/o peuvent être effectuées de manière non bloquante, permettant à fastapi de gérer plus de requêtes en parallèle dans le même thread.
- Une coroutine python est une fonction définie avec **async def** qui, lorsqu'elle est exécutée par une **boucle d'événement**, peut se suspendre (avec **await**) pour céder le contrôle, puis être reprise plus tard.

stratégie avec sqlalchemy + pgsql

1. installer les outils : `pip install asyncpg`

```
# 2. engine
from sqlalchemy.ext.asyncio import create_async_engine,
AsyncSession, async_sessionmaker
DB_URI = "postgresql+asyncpg://Url"
async_engine = create_async_engine(DB_URI)
```

```
# 3. session
AsyncSessionLocal = async_sessionmaker(
    autoflush=True, autocommit=False,
    class_=AsyncSession, bind=async_engine)
```

4. injection de dépendance

```
from typing import AsyncGenerator
async def get_async_db() -> AsyncGenerator[AsyncSession, None]:
    # asyn with replace try / finally car la session est terminée en sortant du contexte
    async with AsyncSessionLocal() as session:
        yield session
    ...
```

5. gestionnaires de contexte

```
import asyncio
from models import Base
from db import async_engine

async def init_db():
    async with async_engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)
    # ...

if __name__ == "__main__":
    asyncio.run(init_db())
```

6. changements clés

- `def` -> `async def`
- `Session` -> `AsyncSession`
- `get_db` -> `get_async_db`
- `db.execute()` -> `await db.execute()`
- `db.commit()` -> `await db.commit()`
- `db.add()` -> reste identique (pas async)
- `db.refresh()` -> `await db.refresh()`
- `db.delete(user)` -> `await db.delete(user)`